

Data Structures and Algorithms

Lab 07 - Queue and Its Applications

Instructor: Sidrat-ul-Muntha

READ IT FIRST

Prior to start solving the problems in this assignments, please give full concentration on following points.

1. **WORKING** – This is individual lab. If you are stuck in a problem contact your teacher, but, in mean time start doing next question (don't waste time).
 2. **DEADLINE** – One Week after assigned this.
 3. **SUBMISSION** – This assignment needs to be submitted in a soft copy.
 4. **WHERE TO SUBMIT** – Please visit your LMS.
 5. **HOW TO SUBMIT** – Rename all .java files according to the task number (e.g., Task1.java, Task2.java). Place all required .java files in a .zip file named 000-00-0000_assignment.zip and submit the ZIP file on the LMS.
-

Learning Objectives

By the end of this lab, students will be able to:

- Implement a Queue using an **array** and perform basic queue operations.
- Implement a Queue using a **singly linked list** with front and rear pointers.
- Reverse a Queue using a **Stack**.
- Implement a **Circular Queue** and reuse empty positions.
- Implement a Queue using **two Stacks** while maintaining FIFO behavior.
- Apply Queues to a real-world application such as **CPU Process Scheduling**.

Task 1: Queue Using Array

Create a class named Queue that implements a **queue using an array**.

The queue should follow the **FIFO (First-In, First-Out)** principle, where the element inserted first is removed first.

Implement the Following Methods

- **enqueue(data)** – Add an element to the rear of the queue.
- **dequeue()** – Remove and return the element from the front of the queue.
- **peek()** – Return the front element without removing it.
- **isEmpty()** – Check whether the queue is empty.
- **isFull()** – Check whether the queue is full.
- **display()** – Display all elements from front to rear.

Demonstration

In the **main()** method create a Queue and demonstrate all the above operations.

Task 2: Queue Using Linked List

Create a class named Queue that implements a **queue using a singly linked list**.

Each node should contain:

- **data** – Stores the element.
- **next** – Stores the reference to the next node.

The Queue should maintain both **front** and **rear** pointers.

Implement the Following Methods

- **enqueue(data)** – Add a new node at the rear of the queue.
- **dequeue()** – Remove and return the element from the front of the queue.
- **peek()** – Return the front element without removing it.
- **isEmpty()** – Check whether the queue is empty.
- **display()** – Display all elements from front to rear.

Demonstration

In the **main()** method create a Queue using a linked list and demonstrate all the above operations.

Task 3: Reverse a Queue Using Stack

Write a method to **reverse the elements of a Queue using a Stack**.

Example

Input Queue:

10 20 30 40 50

Output Queue:

50 40 30 20 10

Requirements

- Use the **Queue** to remove elements.
- Use a **Stack** to temporarily store the elements.
- Move the elements back into the **Queue** after reversing them.
- Do **not** use another Queue or an array for the reversal.

Demonstration

In the main() method:

1. Create a Queue.
 2. Insert several elements into the Queue.
 3. Display the original Queue.
 4. Call the method to reverse the Queue using a Stack.
 5. Display the reversed Queue.
-

Task 4: Circular Queue

Create a class named CircularQueue that implements a **circular queue using an array**.

The circular queue should **reuse the empty positions created after elements are removed**.

Implement the Following Methods

- **enqueue(data)** – Add an element at the rear of the queue.
- **dequeue()** – Remove and return the element from the front of the queue.
- **peek()** – Return the front element without removing it.
- **isEmpty()** – Check whether the circular queue is empty.
- **isFull()** – Check whether the circular queue is full.
- **display()** – Display all elements from front to rear.

Demonstration

In the main() method:

1. Create a CircularQueue with a fixed capacity.
2. Fill the queue with elements.
3. Remove some elements from the front.
4. Add new elements to the queue.
5. Demonstrate how the newly added elements **reuse the empty positions** created by the removed elements.
6. Display the queue before and after performing the operations.

Task 5: Queue Using Two Stacks

Create a class named `QueueUsingStacks` that implements a **Queue using two Stacks**.

The Queue must maintain **FIFO (First-In, First-Out)** behavior even though the underlying data structures are Stacks.

Implement the Following Methods

- **enqueue(data)** – Add an element to the Queue.
- **dequeue()** – Remove and return the front element.
- **peek()** – Return the front element without removing it.
- **isEmpty()** – Check whether the Queue is empty.

Requirements

- Use **exactly two Stack objects**.
- Do **not** use another Queue, array, or linked list for the Queue implementation.
- The Queue must behave according to the **FIFO** principle.

Demonstration

In the `main()` method, demonstrate the following operations:

1. Create a `QueueUsingStacks` object.
2. Enqueue several elements into the Queue.
3. Display or demonstrate the result of the `peek()` operation.
4. Dequeue elements from the Queue and display the removed elements.
5. Enqueue additional elements.
6. Demonstrate that the Queue maintains **FIFO behavior**.
7. Demonstrate the `isEmpty()` operation.

Task 6: CPU Process Scheduling

Create a **CPU Process Scheduling** system using a **Queue**.

Each process should wait in the queue until the CPU is ready to execute it. Processes must be executed in **FIFO (First-In, First-Out)** order.

Requirements

Implement the following methods:

- **addProcess(processName)** – Add a process to the queue.
- **executeProcess()** – Remove and execute the first process in the queue.

- **displayProcesses()** – Display all processes waiting for execution.
- **isEmpty()** – Check whether the process queue is empty.

Example

Add: Process P1

Add: Process P2

Add: Process P3

Queue:

P1 → P2 → P3

Execute:

P1

Queue:

P2 → P3

Demonstration

In the main() method:

1. Create a process scheduling system using a Queue.
2. Add multiple processes to the queue.
3. Display all waiting processes.
4. Execute the processes one by one.
5. Display the queue after each execution.
6. Demonstrate the isEmpty() operation when all processes have been executed.
7. Ensure that all processes are executed in **FIFO order**.